Reactive Following with Five Robots: A Five-Rule Controller in Webots and ROS 2

Lara Dehandschutter
FEUP / Universidade do Porto
up202603432@up.pt

Lennert De Rouck
FEUP / Universidade do Porto
up202602215@up.pt

Pedro Cancela
FEUP / Universidade do Porto
up202400230@up.pt

***Abstract*—We present a purely reactive controller for five differential-drive robots that follow one another in Webots through ROS 2. Every decision uses one current local snapshot: a frontal colour-recognition camera and a frontal LiDAR, both with a 100° horizontal field of view and a 4 m range. The robots share no pose, target identity, map or communication channel, and the controller never retains a target after it disappears. The front and rear halves of each chassis are red and green, so the nearer colour distinguishes a robot facing the observer from one that can be followed. Priority rules select following, approach, head-on avoidance, obstacle escape or a base curve, backed by a separate contact-recovery safety layer. We evaluate the implementation in a 12 m by 12 m Webots arena with four static obstacles. A recorded run completes one full circuit in 165 s; an earlier compact-ring run is retained for comparison, reporting per-robot following occupancy, losses and bearing error. The basic behaviour works, but also exposes important limitations: frontal sensing cannot react to side or rear contacts, recognition is bistable at the camera boundary, and the circuit time is a single observation rather than a statistical benchmark. We discuss these trade-offs and define a repeatable seeded-start protocol for future trials.**

***Index Terms*—reactive robotics, multi-robot following, Webots, ROS 2, LiDAR, robot simulation**

I. Introduction

The task is reactive following: five robots, each deciding only from its own current sensor readings, should aggregate into one or more following chains. The constraints follow classical reactive control in the sense of Brooks' subsumption architecture [1] and Arkin [2]: a priority ordering over situation–action rules, with no world model or planning. The assignment additionally forbids GPS, global localisation, target memory and privileged state, while permitting internal controller state of the PI/PID kind.

Our contribution is threefold: (i) a reproducible five-robot reactive controller with an auditable ROS 2 information boundary, (ii) a geometric LiDAR mounting solution that keeps frontal peers and low walls observable, and (iii) an empirical failure analysis that identifies which limitations come from sensing rather than rule priority. These claims are evaluated in Webots with local regression tests and one recorded obstacle-world run.

One-sentence summary: Five independently controlled robots form and maintain a following train using only instantaneous frontal camera and LiDAR observations, while the experiments expose the limits of that sensing-only architecture.

II. Simulation Model and Sensing

A. Robot

Each robot is a differential-drive cylinder of radius 0.115 m with two powered wheels and an invisible contact bumper of radius 0.146 m. Two robots are in contact when their centres are 0.292 m apart; this number is the reference against which all proximity results below are read.

B. Colour-coded orientation

Two small recognition plaques, red at the front and green at the rear, were replaced by coloured half-shells covering the front and rear halves of the chassis. The reason is angular coverage: a flat plaque shrinks to nothing in projection as a robot turns away, and we measured the leader becoming unclassifiable beyond about 74° between its heading and the observer's line of sight — precisely when a leader is turning, which is when a follower most needs it.

Each half is a separate Webots `Solid` with its own `recognitionColors`. One detail is load-bearing: Webots reports the *origin* of a recognised Solid, so the two halves are offset to $x = \pm 0.060\,\mathrm{m}$. Sharing the chassis centre would make both halves report the same position and front/back would be indistinguishable.

C. Sensors

Sensing is frontal only, as required: a recognition camera and a LiDAR, both 100° and 4 m. The LiDAR mounting deserves comment because it caused most of our early failures and its fix is non-obvious.

Mounted centrally on the roof at $z = 0.155\,\mathrm{m}$ — the natural place, and necessary to clear its own chassis — the LiDAR sits *above* the 0.125 m body top of every other robot. With a narrow vertical field of view, the beam only descends to body height at

$$d \;=\; \frac{0.155 - 0.125}{\tan(\theta_v/2)} \;\approx\; 1.50\,\mathrm{m}, \tag{1}$$

using the deployed $\theta_v = 0.04\,\mathrm{rad}$ vertical field of view, so a peer closer than 1.50 m is invisible. Every collision threshold we might wish to set lies inside that blind radius. Worse, this blindness is unavoidable for a *central* sensor: seeing its own rim at radius r_b without missing a peer at d requires $d > 2r_b$ regardless of θ_v or clearance c. For the body radius this is 0.230 m, below the bumper-defined contact distance of 0.292 m

TABLE I
SIMULATION PLATFORM AND SENSING CONFIGURATION.

Item	Configuration
Middleware	ROS 2 Jazzy, `rclpy`, namespaced topics
Simulator	Webots R2025a
Robot	Five custom differential-drive `SimpleDiffRobot`s
Arena	12 m × 12 m, perimeter walls
Obstacles	Two cylinders, one box, one low barrier
Camera	Recognition, 320×160 px, 100°, 4 m
LiDAR	360 samples, 3 layers, 0.04 rad vertical FOV, 4 m
Update	Webots 32 ms; bridge/controller 50 ms
Noise	No artificial noise; occlusion and clipping retained

— the blind-spot argument concerns body visibility, while the measured contact event comes from the bumper.

The resolution exploits the fact that the LiDAR is now frontal. Mounted at the *front edge* ($x = 0.130\,\mathrm{m}$, outside the body radius), every forward ray's closest approach to the chassis lies behind the sensor, so it can be lowered to $z = 0.115\,\mathrm{m}$: inside the 0.045 m to 0.125 m band of every peer. Robots are then visible at all ranges including contact. A 360° LiDAR cannot do this, because its rearward rays would face its own chassis.

D. Software architecture

Webots runs one controller per robot, which packs a camera/LiDAR snapshot into a UDP datagram. A ROS 2 bridge republishes each as `/Ri/local_sensors`; five independent `SafeFollower` nodes subscribe and publish `/Ri/cmd_vel`. No node subscribes to another robot's topics, which is what makes the no-communication constraint auditable rather than merely asserted.

The implementation uses ROS 2 Jazzy [4] and `rclpy`: the bridge publishes a synchronized snapshot (`sensor_msgs/LaserScan`, camera visibility, contact diagnostics), and controllers publish `geometry_msgs/Twist`. ROS supplies namespaced topics, timers and 20 Hz execution, not localization, navigation, a planner or inter-robot messages. The simulator is Webots R2025a [3], with a custom `SimpleDiffRobot` altered with a frontal camera, a front-mounted three-layer LiDAR, red/green half-shells, differential-wheel actuation and a contact bumper. No sensor noise is injected; uncertainty comes from finite range, occlusion, field-of-view clipping and simulator geometry.

The resulting platform facts are summarized in Table I; the camera and LiDAR are local inputs, while the bumper is a local contact event rather than a ranging sensor.

Figure 1 makes the locality and the absence of a central planner explicit.

III. REACTIVE CONTROLLER

Perception reduces each frame to the nearest red and the nearest green, as (distance, bearing) pairs. The controller is five rules in priority order (Table II).

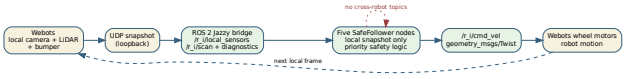


Fig. 1. Information flow from each robot's local Webots devices through UDP and the ROS 2 bridge to its namespaced controller. Dashed feedback is the next local snapshot; the dotted edge emphasizes that controllers exchange no topics.

TABLE II
RULES, IN EVALUATION ORDER. FIRST MATCH WINS.

#	State	Condition	Action
1	`EMERGENCY_*`	contact-range hazard	stop/back off
2	`FOLLOW_HOLD`	rear target at standoff	hold
3	`YIELD_*`	oncoming red / head-on	turn or reverse
4	`AVOID_*`	LiDAR clearance low	turn/back off
5	`FOLLOW`	green rear visible	steer, speed ramp
6	`BASE_CURVE`	otherwise	v_0, ω_{base}
–	`COLLISION_*`	bumper event	recover, then wait

The emergency and avoidance actions use the same conservative escape policy: it reverses at $0.06\,\mathrm{m\,s^{-1}}$ if the LiDAR reads under 0.25 m (`_BACK`), and otherwise rotates on the spot, always clockwise, until the forward cone is clear (`_TURN`). It never advances. Being stateless, it simply stops firing once the way ahead opens, and whichever rule applies next drives off in the direction the rotation happened to reach.

The deployed safety controller also has a contact-triggered layer. A bumper event is local physical feedback: it can report that contact occurred even when the camera and LiDAR could not predict a side or rear collision. It does not provide contact identity or global pose. The recovery backs off, creeps or advances according to current local clearances, then waits before retrying.

The robot-specific v_0 and ω_{base} values in Table III are the only deliberate per-robot differences in the base curve; safety and following thresholds are shared. The LiDAR clearance relation is given in Equation 1.

Three design points proved necessary rather than cosmetic.

a) Red is a comparison, not a detection: Every robot being followed presents a red half somewhere in view, so keying on visibility made followers abandon correct leaders.

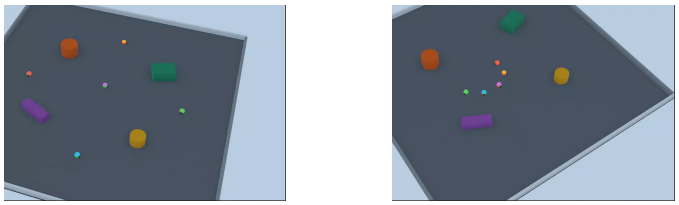
b) Rules 1 and 5 must agree on a direction: They respond to the same physical situation seen through different sensors, so they must not disagree about which way to turn. Section VI-A describes the resulting failure and the shared fix.

IV. EXPERIMENTAL METHOD

The primary trial uses the `big_four_obstacles.wbt` world. The five robots begin in separated poses in the arena, run until a full circuit is completed, and are observed with the same controller and configuration. We define a full circuit as one traversal of the arena's perimeter route that returns the lead robot to its starting side; it is not a closed-loop pose estimate because no global pose is available to the controller.

TABLE III
ROBOT-SPECIFIC BASE-CURVE PARAMETERS FROM THE COMMITTED CONFIGURATION.

Robot	v_0 (m/s)	ω_{base} (rad/s)
R1	0.16	-0.13
R2	0.14	0.09
R3	0.12	-0.08
R4	0.15	0.15
R5	0.13	-0.12



(a) (b)

Fig. 2. (a) Starting configuration in `big_four_obstacles.wbt`: five robots in separated poses around the four static obstacles. (b) The same five robots after forming a single following train.

The static obstacles are a radius-0.42 m, 0.60 m-high cylinder at (-2.40,2.10), a rotated 1.10 m×0.75 m ×0.50 m box at (2.45,2.00), a radius-0.34 m, 0.44 m-high cylinder at (2.30,-2.20), and a 1.35 m×0.45 m×0.36 m barrier at (-2.45,-2.10), in metres.

Figure 2(a) shows this starting configuration. We record circuit completion time, per-robot following duration, mean absolute bearing error while following, target losses (transitions out of a following state), and bumper contacts. The offline regression suite exercises safety priority, marker classification, LiDAR geometry and corner recovery with synthetic snapshots; it does not replace a Webots physics trial.

The protocol is repeatable but not yet statistically complete. The seeded script `tools/randomize_starts.py` samples five positions within ± 2.6 m, enforces a minimum pairwise gap of 2 m, and samples headings uniformly from $[0, 2\pi)$; the same seed reproduces a layout. For a valid comparison, the same seed is used across controller variants and different seeds are used across repetitions. No noise or random sensor dropout is added, although contact recovery includes a random 0 s to 1 s restart delay. This protocol was not executed for the present paper: only one obstacle-world run has been timed, so the reported 165 s is descriptive, not a mean or claim of generality. Webots pose traces were not saved, so no trajectory plot or current-run occupancy statistics are fabricated.

V. RESULTS

Table V gives historical per-robot figures from a 291.2 s compact-ring observation window; it is not a circuit time and is not the result of the current four-obstacle experiment. The current run completed one defined perimeter circuit in 165 s.

Time-weighted occupancy was `FOLLOW` 71.9%, `BASE_CURVE` 27.5%, and all avoidance states together below 1% (`AVOID_RED` 0.3%, `ESCAPE_TURN` 0.2%, `AVOID_WALL` 0.1%).

TABLE IV
COMPARISON OF THE TWO RECORDED OBSERVATIONS. THE WORLDS AND STARTS DIFFER, SO THIS IS DESCRIPTIVE RATHER THAN A CONTROLLED COMPARISON.

Trial	Arena/obstacles	Observation	Recorded metric
Compact ring (historical)	compact / not comparable	291.2 s	71.9% FOLLOW
Current obstacle world	12 × 12 m / four	165 s	1 perimeter circuit

TABLE V
PER-ROBOT RESULTS, 291.2 S RUN.

	Following	%	Losses	Mean $\lvert\beta\rvert$ (rad)
R1	290.9 s	99.9	2	0.629
R2	291.1 s	99.9	1	0.702
R3	44.7 s	15.3	2	0.420
R4	282.8 s	97.1	3	0.729
R5	137.2 s	47.1	6	0.451

In the historical compact-ring run, formation proceeded in five phases: 82 s of scattered search; 34 s of first acquisitions; an 11 s congestion incident; 59 s of re-forming; and then a train that persisted for the remaining 105 s with no logged avoidance event. The end state was the intended one — R3 leading on its base curve, R1, R2, R4 and R5 following — which is why R3's low following percentage is a success rather than a failure: the leader of a train is, by definition, not following anything. Figure 2(b) shows the five robots after the train has formed.

The unweighted mean of the five per-robot bearing errors is 0.586 rad; weighting by each robot's following time gives 0.644 rad. Neither should be read as tracking error. In a five-robot ring the leader is geometrically about 36° off-axis, so the measured values are dominated by formation geometry rather than a zero-error target.

The comparison is intentionally limited: the two trials use different layouts and worlds, so the shorter current time is not a controller improvement. The useful result is the combination of circuit completion, the per-robot table and an explicit account of unmeasured variance. The main advantages are low computational complexity, auditable local information flow and graceful rule-level safety; the main limitations are the single-run benchmark, absent artificial noise, frontal-only sensing and no quantitative success rate over random seeds.

Beyond this timed circuit, the controller was also exercised on several arenas from `tools/generate_random_world.py`, which randomizes all five starting poses and zero to five obstacles per seed in the same 12 m×12 m arena. In every run observed the five robots eventually formed a single chain; the same mechanisms discussed in Section VI — bistable edge-of-view detection and open-loop corner or wall behaviour — were what delayed convergence. This is a qualitative, non-tabulated observation, not a statistical batch.

VI. QUIRKS, SHORTCOMINGS AND LIMITATIONS

Each item below states the mechanism, the evidence for it, what it costs in behaviour, and what removing it would require.

Several are properties of the sensing envelope and cannot be fixed inside the control rules at all.

A. Close-range escape: two rules that disagreed

The original close-range rules could disagree when a peer alternated between camera-visible and anonymous LiDAR obstacle. At 0.15 m, a lateral offset of 1 cm changes bearing by about 3.8° and 8 cm by about 28.1°, making the $\pm 50^\circ$ camera boundary easy to cross. The deployed controller unifies these cases: it turns clockwise without advancing, reverses only when local clearance permits, and uses a $\pm 45^\circ$ obstacle cone. This removes left/right chatter but leaves open-loop corner behaviour.

Concretely, the rotation stops once the forward cone clears, often leaving the robot heading *parallel* to the wall rather than away from it. Whichever rule fires next — typically `BASE_CURVE` with its own fixed ω_{base} — has no knowledge of why that rotation happened, so if its curve points back at the same wall the robot grazes along it instead of separating. Escape and base curve are independent, stateless rules by design; fixing this would require passing the exit heading forward, exactly the kind of state the instantaneous-snapshot constraint is meant to avoid.

B. Bistable detection at the edge of the field of view

A target sitting near $\pm 50^\circ$ is alternately inside and outside the camera cone from one frame to the next, because the bearing fluctuates by more than the margin separating it from the boundary. The controller has no notion of a marginal detection: a target is either present or absent, and different rules fire in the two cases.

At the field-of-view boundary, one frame can classify a peer as a red robot and the next as no robot. This produced short avoidance oscillations in the diagnostic logs. Hysteresis would reduce the chatter, but retaining it would introduce target memory, so the present implementation accepts this limitation to preserve the assignment's instantaneous-snapshot constraint.

C. Prediction versus contact recovery

Both active ranging and recognition are frontal and span 100°, so a robot alongside or behind is indistinguishable from empty space until geometry or the bumper changes — "undetectable" applies to pre-contact side/rear perception, not to the bumper itself. A bumper event gives a local contact signal with no identity, direction or pose; the safety layer uses current LiDAR clearances and contact state to back off, creep or advance, then waits. This recovers from some side/rear contacts but cannot anticipate or avoid them; closing that gap needs an actual side or rear sensor.

D. Formation geometry is coupled to the turn cap by hand

For five robots on a ring, spacing and radius satisfy $s = 2R\sin(\pi/5) = 1.176R$, while the camera reads 0.19 m less than centre spacing. Thus a chosen standoff implies a ring radius that must be compatible with the turn-rate cap, the same curvature trade-off seen in pure-pursuit tracking [5]. Earlier inconsistent values forced the formation to the speed floor; the current parameters avoid that case, but the code still lacks an assertion to catch future retuning errors.

A related failure appears when a robot's own ω_{base} is large: most randomized starts still converge, but if that robot ends up leading rather than following, its sharper base curve can cut back across its own followers' path and break the train on contact, rather than merely forcing the speed floor.

E. Tests are logic-level, not physics-level

The repository contains regression tests for the safety cascade, marker perception and corner/obstacle decisions. These tests use synthetic sensor snapshots and therefore do not validate Webots contact physics, camera occlusion or the measured circuit time. The tests are useful for detecting contradictory steering decisions, but the obstacle-world result still requires an actual Webots run.

VII. Conclusion

A five-rule reactive controller, acting on one frontal snapshot at a time, enabled the five robots to complete a full circuit of the current large world in 165 s. The world contains four static obstacles and uses separated starting poses rather than the earlier compact ring. The rules were cheap; the difficulty was almost entirely in the interaction between the sensing envelope and the geometry of a multi-robot formation. The dominant failure mode we did not eliminate — bistable detection at the edge of the field of view — is a property of where the sensor stops, not of what the controller decides.

The proposal is therefore a transparent baseline, not a complete multi-robot navigation solution: each robot is independent, the ROS interfaces make the information boundary inspectable, and the priority rules are easy to test with synthetic snapshots, but there is no side or rear sensing, no target hysteresis or identity, no injected noise model, and no repeated statistical evaluation yet. The next experiment should run the seeded protocol over multiple layouts and report success rate, contacts, completion time and confidence intervals, to test whether the observed train is robust rather than merely reproducible in one favourable run.

References

[1] R. A. Brooks, "A robust layered control system for a mobile robot," *IEEE J. Robot. Autom.*, vol. 2, no. 1, pp. 14–23, 1986.

[2] R. C. Arkin, *Behavior-Based Robotics*. Cambridge, MA, USA: MIT Press, 1998.

[3] O. Michel, "Webots: Professional mobile robot simulation," *Int. J. Adv. Robot. Syst.*, vol. 1, no. 1, pp. 39–42, 2004.

[4] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, 2022.

[5] R. C. Coulter, "Implementation of the pure pursuit path tracking algorithm," Robotics Institute, Carnegie Mellon Univ., Tech. Rep. CMU-RI-TR-92-01, 1992.